# COMP0005 – ALGORITHMS

Week 4 – 04 Feb 2025

**Prof Licia Capra**

**Dept of Computer Science**

# OUTLINE

- [This] Week Preview
- [Last] Week Recap
- Exam-style Exercises on Sort

# [THIS] WEEK PREVIEW

- Topic 2 – Sorting algorithms
  - 2.3 Quick Sort
  - 2.4 Heap Sort
- Self-practice: Sorting algorithms
- Lab session: TA Lab – Week 4
- Muddiest point (Weeks 3 & 4 - Sort)

  *Closes Wed 5th Feb 2025 @ 23:59*

# [LAST] WEEK RECAP

- Selection Sort ← $O(N^2)$ *in all cases*
  - Scan left to right
  - Find min element
  - **Swap** min to its final position
- Insertion Sort ← $O(N^2)$ vs $O(N)$ (worst vs best case)
  - Scan left to right
  - **Shift** element to preserve ascending order
- Merge Sort ← $O(N \log N)$ *in all cases*
  - Divide & Conquer

# EXAM-STYLE EXERCISES

- Understanding
  - Main Summer Exam 2022/23 – Q1(b)
  - Main Summer Exam 2021/22 – Q1(a)
  - Main Summer Exam 2021/22 – Q1(b)

- Problem Solving / Design
  - Example: Anagrams

# 2022/23 Main Exam – Q1(b)

Suppose you have the following array `a[]` of 9 elements (indexed from 0 to 8) that you need to sort using the Mergesort algorithm:

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| `a[]` | 18 | 17 | 11 | 13 | 14 | 19 | 15 | 16 | 12 |

Consider the recursive implementation of the Mergesort algorithm seen in lectures, with auxiliary array `aux[]` used to perform the merge step. Give the complete sequence of recursive calls to `sort(a, aux, low, high)` and `merge(a, aux, low, mid, high)` in the exact order they would be called to completely sort the above array. In providing your answer, use Mergesort with no optimisation for short arrays; write down the list of `sort()` and `merge()` calls as an ordered sequence (do not depict a tree of calls). At each step, there is no need to show the content of the arrays `a[]` and `aux[]`; however, you should clearly state on what portion of the original array `a[]` each recursive call is working on (that is, what values the indices low, mid, high have). Start from a call to `sort(a, aux, 0, 8)`. *[10 marks]*

# 2022/23 Main Exam – Q1(b)

| a[] | 18 | 17 | 11 | 13 | 14 | 19 | 15 | 16 | 12 |
|---|---|---|---|---|---|---|---|---|---|

```
sort(a[], aux[], lo, hi):
    if (hi <= lo) return;

    mid = lo + (hi - lo) / 2;

    sort(a, aux, lo, mid);
    sort(a, aux, mid+1, hi);
    merge(a, aux, lo, mid, hi);
```

# 2022/23 Main Exam – Q1(b)

| a[] | 18 | 17 | 11 | 13 | 14 | 19 | 15 | 16 | 12 |
|---|---|---|---|---|---|---|---|---|---|

```
merge(a[], aux[], lo, mid, hi):
    for (k = lo; k <= hi; k++)
        aux[k] = a[k];

    i = lo;
    j = mid+1;
    for (k = lo; k <= hi; k++)
         if (i > mid) {a[k] = aux[j]; j++;}
         else if (j > hi) {a[k] = aux[i]; i++;}
         else if (aux[j] < aux[i]) {a[k] = aux[j]; j++;}
         else  {a[k] = aux[i]; i++;}
```

# 2022/23 Main Exam – Q1(b)

| a[] | 18 | 17 | 11 | 13 | 14 | 19 | 15 | 16 | 12 |
|---|---|---|---|---|---|---|---|---|---|

```
merge(a[], aux[], lo, mid, hi):
   for (k = lo; k <= hi; k++)
      aux[k] = a[k];

    i = lo;
    j = mid+1;
    for (k = lo; k <= hi; k++)
         if (i > mid) {aux[k] = a[j]; j++;}
         else if (j > hi) {aux[k] = a[i]; i++;}
         else if (a[j] < a[i]) {aux[k] = a[j]; j++;}
         else  {aux[k] = a[i]; i++;}
```

# 2021/22 Main Exam – Q1(a)

Suppose you have the following initial array:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 7 | 1 | 16 | 10 | 4 | 12 | 11 | 8 |

What sorting algorithm(s) will encounter:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 4 | 7 | 10 | 16 | 12 | 11 | 8 |

during the sorting process? In answering this question, consider each of the following sorting algorithms in turn: Insertion Sort, Selection Sort, Merge Sort. To fully document your answer, provide the intermediate sorting configurations encountered by each algorithm, up to the point where it is clear that pattern will (or will not) be encountered. *[15 marks]*

# 2021/22 Main Exam – Q1(a)

Suppose you have the following initial array:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 7 | 1 | 16 | 10 | 4 | 12 | 11 | 8 |

What sorting algorithm(s) will encounter:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 4 | 7 | 10 | 16 | 12 | 11 | 8 |

during the sorting process?

```
#insertion sort
for i in range(1,len(A)):
    j=i
    while (j>0) and A[j-1]>A[j]:
        swap(A[j], A[j-1])
        j = j-1
```

# 2021/22 Main Exam – Q1(a)

Suppose you have the following initial array:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 7 | 1 | 16 | 10 | 4 | 12 | 11 | 8 |

What sorting algorithm(s) will encounter:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 4 | 7 | 10 | 16 | 12 | 11 | 8 |

during the sorting process?

```
#insertion sort
for i in range(1,len(A)):
    j=i
    while (j>0) and A[j-1]>A[j]:
        swap(A[j], A[j-1])
        j = j-1
```

```
#selection sort
for i in range(0,len(A)):
    min = i
    for j in range(i+1,len(A)):
        if (a[j] < a[min]):
            min = j
    swap(A[i], A[min])
```

# 2021/22 Main Exam – Q1(b)

UCL Portico contains past and present students' records for millions of students. Assume each record contains the following fields: the student's surname (string), their first name (string), a unique student identification number (integer), the programme of study they enrolled in (string), and the year of enrolment (integer). For example, a single record would look like:

| Surname | Name | ID | Programme of Study | Year |
|---|---|---|---|---|
| Capra | Licia | 123456 | PhD Computer Science | 2000 |

What sorting algorithm is most appropriate to use to sort Portico students' records? Note that sorting can take place on any field; furthermore, consecutive sorting steps may be used (e.g., first sort by programme of study, then by year). In giving your answer, consider four possible algorithms: Selection Sort, Insertion Sort, Merge Sort and Quick Sort. Thoroughly discuss their pros and cons in this specific context. *[9 marks]*

# EXAMPLE – Anagrams

Given a list of words, design an efficient algorithm to group all anagrams together. Words w1 and w2 are said to be anagrams if by rearranging the letters of w1, you can get w2 using all the original letters of w1 exactly once. For example, given the list of words:

```
[eat, tea, part, ate, trap, pass]
```

your algorithm should output:

```
[eat, tea, ate]
[part, trap]
[pass]
```

Use pseudocode to illustrate your answer. Discuss the computational cost of your solution.

# SORT (part I) – TAKE THE TEST …

https://tinyurl.com/86emtt9r